END-TO-END PIANO PERFORMANCE-MIDI TO SCORE CONVERSION WITH TRANSFORMERS

Tim Beyer
Technical University of Munich
tim.beyer@tum.de

Angela Dai
Technical University of Munich
angela.dai@tum.de

ABSTRACT

The automated creation of accurate musical notation from an expressive human performance is a fundamental task in computational musicology. To this end, we present an end-to-end deep learning approach that constructs detailed musical scores directly from real-world piano performance-MIDI files. We introduce a modern transformer-based architecture with a novel tokenized representation for symbolic music data. Framing the task as sequence-to-sequence translation rather than note-wise classification reduces alignment requirements and annotation costs, while allowing the prediction of more concise and accurate notation. To serialize symbolic music data, we design a custom tokenization stage based on compound tokens that carefully quantizes continuous values. This technique preserves more score information while reducing sequence lengths by $3.5\times$ compared to prior approaches. Using the transformer backbone, our method demonstrates better understanding of note values, rhythmic structure, and details such as staff assignment. When evaluated end-to-end using transcription metrics such as MUSTER, we achieve significant improvements over previous deep learning approaches and complex HMM-based state-of-the-art pipelines. Our method is also the first to directly predict notational details like trill marks or stem direction from performance data. Code and models are available on GitHub.

1. INTRODUCTION

Creating structured musical scores from human performance recordings is a challenging task with a significant number of downstream applications in areas such as alignment [1], score-following, education, and archiving.

Human performances are typically represented as performance-MIDI (P-MIDI) files, as they can be easily recorded from MIDI instruments or generated from audio by automated transcription systems [2, 3]. In contrast, high-quality scores in standard sheet music formats such as MusicXML [4] are much less commonly available and generally require creation by human experts.

Performance-MIDI-to-Score conversion (PM2S) is complex, encompassing several lower-level tasks like note value prediction, tempo regression, rhythm quantization, voice assignment, and typesetting details, including ornaments and note stems. As a result, PM2S and its sub-tasks have remained an active research topic and popular application of computational methods for over 30 years [5].

While early approaches relied on classical modeling and hand-crafted processing [6], research gradually shifted towards statistical methods based on Hidden Markov Models (HMMs) augmented with heuristics.

Cogliati et al. [7] run an HMM-based meter estimation [8] with beat-snapping heuristics to quantize note timings, before outputting LilyPond [9] notation. HMM variants were also used for estimating staff placement [10] and rhythm quantization [11, 12], where the outputs of multiple models are merged into a final prediction with improved accuracy. To complement onset timing quantization, a method for note value recognition based on Markov random fields was introduced by Nakamura et al. [13]. Building upon these advances, Shibata et al. [14] combined prior systems with hand-crafted non-local statistics to improve estimates of global attributes such as piece tempo and time signatures. While these approaches yield state-of-the-art performance, they are composed of a complex web of interdependent components, rely on human-designed priors, and are not trained end-to-end. Thus, more recent work has attempted to tackle PM2S using deep learning.

So far, the scarcity of high-quality labeled data has limited its use to scenarios with cheaper labels, leading to a focus on synthetic data [15, 16], sub-tasks like pitch-spelling [17], note value quantization and voicing [18], or beat tracking [19, 20]. Beat tracking, in particular, has seen significant progress by combining CRNNs with beat in-filling via dynamic programming [19]. The CRNN also predicts other score attributes such as key and time signatures. Unfortunately, beat-tracking makes overly restrictive assumptions about the regularity of the underlying performance data and struggles with real-world human recordings.

Another challenge is the representation of symbolic music data for machine learning models. Monophonic sequences are often captured as Lilypond [9, 15], ABC [21], Humdrum-derived [16, 22], or custom CTC-friendly [16] character sequences. For polyphonic data, piano rolls [2, 23], MIDI-derived tokens [24–26], and custom MusicXML tokens [27] are the most common representations. To create more compact encodings, Zeng et al. [28] and Dong et al. [29] use compound tokens and represent MIDI attributes in separate streams, shortening sequence lengths.

© T. Beyer and A. Dai. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** T. Beyer and A. Dai, "End-to-End Piano Performance-MIDI to Score Conversion with Transformers", in *Proc. of the 25th Int. Society for Music Information Retrieval Conf.,* San Francisco, United States, 2024.

Our proposed approach continues the progression of PM2S systems towards end-to-end learned approaches and overcomes several limitations of prior systems based on deep learning, making the following key contributions:

- We cast PM2S as an end-to-end sequence-to-sequence translation task, developing a transformer to enable accurate prediction of global attributes (e.g., meter) that require understanding of long-term dependencies.
- Relaxed annotation requirements compared to prior deep learning methods, using only beat-level alignment for training. We can additionally leverage unmatched MusicXML data without corresponding P-MIDI.
- We introduce a compact and extensible tokenization scheme for P-MIDI and MusicXML data, allowing the backbone model to directly translate tokenized P-MIDI into MusicXML tokens and enabling the generation of detailed score features such as ornaments.
- We demonstrate superior performance on quantitative error metrics like MUSTER, with our approach surpassing prior deep learning models and the highly optimized, complex state-of-the-art.

2. METHODOLOGY

2.1 Task definition

Our end-to-end PM2S system directly converts an unstructured P-MIDI file into a highly readable MusicXML score. P-MIDI files only contain information about note timing (onsets, offsets), pitch, and velocity. The input to a PM2S system is thus defined by the following sequence:

$$\mathbf{X} = \{(p_i, o_i, d_i, v_i)\}_{i=1}^{N_{\text{perf}}}, \tag{1}$$

with MIDI pitch p_i, onset o_i and duration d_i in seconds, and velocity v_i for each of the N_{perf} performance notes.

In contrast to existing methods, which often cast PM2S as a note-wise classification task [18,19], we do not assume a one-to-one correspondence between notes in the performance and the score. This is crucial in scenarios with trills or misplayed notes, where one-to-one matchings are impossible. Consequently, we predict a new output note sequence from scratch that includes a full set of MusicXML attributes for each note in the score:

$$\mathbf{Y_q} = \{(p_j, mo_j, md_j, ml_j)\}_{j=1}^{N_{\text{score}}} \tag{2}$$

$$\mathbf{Y_v} = \{(h_j, vo_j)\}_{j=1}^{N_{\text{score}}} \tag{3}$$

$$\mathbf{Y_o} = \{(t_j, s_j, sd_j, g_j, a_j)\}_{j=1}^{N_{\text{score}}} \tag{4}$$

$$\mathbf{Y} = (\mathbf{Y_q}, \mathbf{Y_v}, \mathbf{Y_o}), \tag{5}$$

where $\mathbf{Y_q}$ comprises attributes related to pitch p_j and quantized timings for the musical onset time mo_j, musical duration md_j, and measure length ml_j. $\mathbf{Y_v}$ collects vertical positioning information, such as staff placement/hand h_j, and MusicXML voice number vo_j. Finally, $\mathbf{Y_o}$ covers performance annotations, ornamentation, and typesetting details like trill t_j, staccato s_j, stem direction sd_j, grace note g_j, and accidentals a_j. Predicting these additional attributes enables creating more concise and accurate notation. $\mathbf{X}$ and $\mathbf{Y}$ are sorted by ascending onset/offset, pitch, and duration, yielding a unique serialized representation even for complex polyphony.

2.2 Tokenization scheme

To efficiently represent input $\mathbf{X}$ and output $\mathbf{Y}$, we introduce a systematic tokenization for P-MIDI files and MusicXML scores. The key objective of a tokenization algorithm is to retain as much information from the original sequence as possible within a compact sequence length and vocabulary size. Thus, we adopt a parallel token stream paradigm [28]; a separate token stream is constructed for each of the four input attributes given in Eq. (1) and the eleven output attributes in Eq. (5). As a result, each note occupies only one timeslot. The final vocabulary sizes and parameter ranges are shown in Table 1.

For P-MIDI, we adopt a strategy similar to [19] and use 128 pitch tokens, 8 quantized velocity tokens, and quantize delta onsets and durations into 200 buckets. To achieve high resolution for small values while covering times up to 8 seconds without clipping, we apply a log-transform before bucketing onsets and durations, implementing a continuous version of multi-resolution quantization [28].

We pay particular attention to the MusicXML tokenization. While binary and categorical attributes, such as stem direction and staff assignment, are easily tokenized, continuous values like onsets and durations demand more care. The encoding of musical timing and positioning significantly impacts score quality. To find a good trade-off between vocabulary size and the ability to correctly represent common note durations and onsets, we conducted a search over bucket sizes. Consequently, we opt to quantize musical time into $\frac{1}{24}$th fractions, diverging from previous approaches, which rely on powers of 2 or smaller denominators [18, 28, 30]. Our parameterization accurately represents 98.6% of notes in the ASAP dataset with 97 tokens, compared to just 85.4% using powers of 2 up to 256.

Encoding absolute musical onsets into tokens poses another challenge. Direct quantization of absolute positions is infeasible due to the large range of positions required, while delta encoding similar to MIDI quickly leads to drift issues and misalignment with measure boundaries and the

Parameter	N_{vocab}	Range/Values
Input Parameters		
Pitch (p_i)	128	$[0, 127]$
Onset (o_i)	200	$[0, 8]$
Duration (d_i)	200	$[0, 8]$
Velocity (v_i)	8	$[0, 127]$
Output Parameters		
Pitch (p_j)	128	$[0, 127]$
Musical Onset (mo_j)	145	$[0, 6]$
Musical Duration (md_j)	97	$[0, 4]$
Measure Length (ml_j)	146	$[0, 6] \cup \{\text{false}\}$
Hand/Staff (h_j), Trill (t_j), Grace (g_j), Staccato (s_j)	2 each	boolean
Voice (vo_j)	8	$[1, 8]$
Stem (sd_j)	3	{up, down, none}
Accidental (a_j)	6	{𝄫, ♭, ♮, ♯, 𝄪, none}

Table 1. Parameter specifications for input/output representations. The rightmost column details the range or set of representable values for each attribute. Continuous values outside the range are clipped before tokenization.

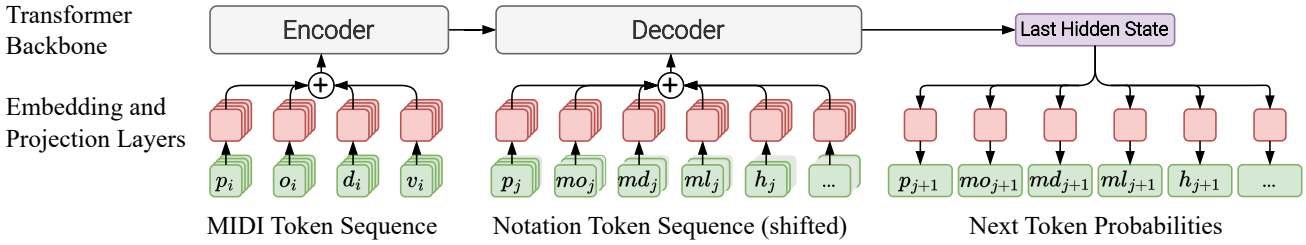


Figure 1. Model architecture overview. We use a standard Roformer encoder-decoder model [31] with custom token embedding and projection layers. Each token stream is embedded separately, then a constant-size shared embedding is created via summation. The backbone model architecture remains unchanged compared to models applied to NLP or other sequence-to-sequence learning tasks. In this illustration, depth symbolizes the time direction.

musical grid. Thus, we adopt a hybrid approach representing absolute positions using two tokens; mo_j encodes the note's position relative to the start of the current measure, and ml_j stores the preceding measure's length for the first note in each measure or is set to `false` otherwise. Combined, mo_j and ml_j enable the reconstruction of absolute musical times, including correct bar lines and most time signatures. During score creation, bar lines are used to split and tie notes crossing measure boundaries, recovering most ties, and rests are added to fill any gaps in voices.

Since input and output streams are not necessarily the same length, we also insert space tokens (sp_j) where required for alignment (see also Section 3.1.2). The space tokens differ from typical end-of-sequence padding or masking tokens in Transformers, as they are predicted during inference and attention to these positions is not masked.

2.3 Model architecture

By adopting a unified autoregressive Transformer encoder-decoder model [32] to directly translate tokenized P-MIDI into MusicXML tokens (as depicted in Figure 1), we diverge from existing deep learning models for PM2S, which used subtask-specific LSTMs [33] or CRNNs. Our choice is driven by the transformer's ability to scale to large datasets and to handle long-range dependencies, which are crucial for predicting piece-wide attributes like meter.

To interface with parallel token streams, our model introduces custom embedding and projection modules. First, each attribute-specific token stream is mapped into a constant-size 512-dimensional embedding space. The results are then summed and normalized using LayerNorm [34] to form a constant-size shared embedding, independent of the token stream count.

The backbone model itself follows the original architecture described by Vaswani et al. [32] and consists of symmetrically arranged encoder and decoder stacks. Each stack comprises four layers, eight attention heads, and a model dimension of 512. To optimize performance, we adopt rotary positional encodings [31], pre-norm [35], and SwiGLU activations [36] with an inner dimension of 3072 for the position-wise feed-forward network. At the end of the decoder, a set of linear layers projects the final hidden state into one output logit distribution per token stream.

2.4 Training & inference details

To optimize our model parameters, we break down the loss computation into two stages and first compute per-timestep losses $\mathcal{L}_j$, before summing along the sequence position. At each timestep j, our model performs 12 separate classification tasks, one for every token stream in $\mathbf{Y}$ and one for the space token sp_j.

We compute the cross entropy (CE) loss for each output token stream y and the space token stream. For timesteps with spacing token sp_j, the loss is calculated only for the space token stream since the labels for all other tokens are undefined. The full loss computation is thus:

$$\mathcal{L}_{y,j} = \mathrm{CE}\left(\hat{y}_j, y_j\right) \tag{6}$$

$$\mathcal{L}_j = \begin{cases} \mathrm{CE}\left(\hat{sp}_j, 1\right) & \text{for } sp_j = 1, \\ \sum_{y \in \mathbf{Y}} \mathcal{L}_{y,j} + \mathrm{CE}\left(\hat{sp}_j, 0\right) & \text{otherwise.} \end{cases} \tag{7}$$

$$\mathcal{L} = \sum_{j=1}^{N_{\text{score}}} \mathcal{L}_j. \tag{8}$$

We train our model for 40,000 steps using the AdamW optimizer [37]. The learning rate follows a cosine learning rate decay schedule with linear warmup over the first 4,000 steps to a maximum learning rate of 3e-4. Gradients are clipped to a maximum value of 0.5. We use a batch size of 32 and the training sequence length is 512 timesteps.

To parallelize training, transformers are often trained with teacher forcing. However, exposure bias [38] can lead to lower-than-expected performance at inference time, especially in the low-data regime. We find that heavy dropout [39] during training to expose the model to only 25% of the preceding output tokens addresses this problem.

During inference, we employ greedy top-1 decoding as it provides better performance than alternatives. To handle songs with more than 512 notes, we partition the input into chunks of 512 notes each, ensuring a 64-note overlap between consecutive segments. Sufficient overlap eliminates abrupt changes at the segment boundaries and is essential for generating temporally coherent scores.

3. EXPERIMENTS

3.1 Data

3.1.1 Datasets

Unlike prior PM2S systems [14, 18, 19], we do not use the MAPS [40] dataset in our experiments, as performance data contained therein is not representative of real-world P-MIDI [1] . Furthermore, its musical scores are only available

[1] The underlying data is score-derived and has been manually adjusted to represent aspects of performance and score at the same time. This leads

in the MIDI format, which lacks representational capacity compared to MusicXML. Thus, MAPS scores do not effectively capture many aspects of musical notation.

To overcome these limitations, we use the ASAP dataset [41] for training and evaluation. It contains 1067 pieces of P-MIDI recorded from expert piano performances and corresponding high-fidelity MusicXML scores. Performance and scores are aligned with beat-level annotations, which are significantly cheaper to obtain than note-level alignments. We also observed that, on average, MusicXML scores contain 2.6% fewer notes than associated P-MIDI. These discrepancies are typically caused by misplayed notes and trills, again highlighting the importance of a flexible approach that is not reliant on one-to-one correspondences and can handle a wide variety of notation features.

After manually inspecting the dataset, we reject 100 instances due to poor alignment or data corruption, leaving 967 performances. We perform only minimal preprocessing, focusing on removing non-sounding notes from the score. This includes merging tied notes into a single, longer note and removing notes with the MusicXML `print-object=no` attribute, as they would not be visible to a human performer.

To guarantee non-overlapping sets with robust evaluation across all composers in the dataset, dataset splits are created using the following procedure:

For each composer, we select one piece as a test piece and use all performances of this piece for the test set, yielding 59 instances. 90% of all remaining pieces are used in the training set and 10% in the validation set. Table 2 shows the resulting full dataset split statistics.

To complement this labeled dataset, we also construct an unpaired dataset consisting of 58,646 public domain MusicXML files from Musescore, without corresponding P-MIDI. These scores are filtered for overlap with the labeled dataset to avoid data leakage.

Dataset	Train	Validation	Test	Total
Performances	822	86	59	967
Distinct pieces	176	16	14	206
P-MIDI Notes (10^3)	2510	300	220	3030
Score Notes (10^3)	2462	295	215	2972

Table 2. Dataset statistics for ASAP [41] after excluding instances with mismatched annotations.

3.1.2 Training batch construction

All training batches consist of 32 sequences of 512 notes each, equally split between labeled and unpaired datasets. To sample instances from these heterogeneous datasets, we adopt two different procedures.

Labeled data. We first use the beat-level correspondences to coarsely align input and output sequences by sorting notes into inter-beat intervals according to their onset time. Although this correspondence is exact for the MusicXML score data, human performances introduce variations to the P-MIDI data, causing some notes to not align perfectly with annotated beats. As a result, performance notes that occur shortly before the annotated beat time may musically belong into the next inter-beat interval and vice-versa. To solve this issue, we follow a greedy optimization strategy that minimizes mismatched pitches between performance and score in each beat interval. If a performed note occurred within 50ms of a beat, and moving it to the previous/next inter-beat-interval reduces the number of mismatched pitches in both intervals, the move is performed. Where necessary for alignment, we add spacing tokens (sp_j) at the end of inter-beat intervals. Given correct beat annotations, this procedure yields good alignment even in non-trivial situations like trills, where multiple MIDI notes correspond to just one MusicXML note.

Unpaired data. In this case, only MusicXML data is available. This could be used to simply pre-train the decoder stack in an autoregressive fashion; however, we found this procedure to be ineffective. We thus aim to incorporate the encoder into the training process and construct a surrogate input token stream by reusing the output pitch tokens p_j as input for the encoder model p_i and mark the input sequence using conditioning tokens c_i. All other input tokens (o_i, d_i, and v_i) are masked. As demonstrated in Section 3.4, this significantly enhances the effectiveness of training on unpaired data. Without input timing and velocity streams, the model has far less information to make predictions. To make the learning objective more feasible, we decrease the prior-token dropout probability to 50% (compared to 75% for paired data), improving training efficiency without compromising inference time behavior (see also Section 2.4). Similar to conditioning masks in diffusion models, we also feed a binary token (c_i) to the encoder which indicates that no real P-MIDI conditioning information from the labeled dataset is provided, resolving ambiguity about whether input tokens are masked/dropped out or simply not available. The addition of this token improves the effectiveness of training on unlabeled data (see Table 6). When training on labeled data and during inference, its embedding is set to $\mathbf{0}$ and can thus be omitted.

3.1.3 Data augmentation

During training, four types of data augmentation are used to combat overfitting:

- **Transposition:** Shift all pitches in the input and output up or down by up to 12 semitones; notes falling outside the MIDI pitch range are shifted inward by one octave. Accidentals are modified accordingly, following [17].
- **Global tempo:** Change the timing data of the input MIDI notes by a factor of $\lambda \sim \mathcal{U}(0.8, 1.2)$.
- **Duration jitter:** To simulate human performance variations, performed note durations are additionally rescaled by a small amount of noise $\sim \mathcal{U}(0.95, 1.05)$.
- **Onset jitter:** All between-note intervals of the input MIDI are changed according to $\tilde{o}_{i+1} - \tilde{o}_i = (o_{i+1} - o_i) \cdot \mathcal{N}(1, 0.05^2)$.

to information leakage as highly regular onset/offset alignment remains in the 'performance' data. Details of the laborious alignment process are available at `http://www.piano-midi.de/technic.htm`.

Method	MUSTER [14]						ScoreSimilarity [27, 42]					
	$\mathcal{E}_{\text{p}}$	$\mathcal{E}_{\text{miss}}$	$\mathcal{E}_{\text{extra}}$	$\mathcal{E}_{\text{onset}}$	$\mathcal{E}_{\text{offset}}$	$\mathcal{E}_{\text{avg}}$	$\mathcal{E}_{\text{miss}}$	$\mathcal{E}_{\text{extra}}$	$\mathcal{E}_{\text{dur.}}$	$\mathcal{E}_{\text{staff}}$	$\mathcal{E}_{\text{stem}}$	$\mathcal{E}_{\text{spell.}}$
Neural Beat Tracking (improved) [19]	**2.02**	6.81	9.01	68.28	54.11	28.04	17.10	17.67	66.98	6.86	-	9.71
MuseScore [43]	2.41	7.35	9.64	47.90	49.44	23.35	16.17	16.74	55.23	21.87	29.87	9.69
Finale [44]	2.47	10.10	13.46	31.85	45.34	20.64	14.72	16.43	53.35	21.79	26.74	15.34
HMMs + Heuristics (J-Pop) [14]†	2.09	**6.38**	8.67	25.02	29.21	14.27	10.80	11.39	71.38	-	-	-
HMMs + Heuristics (classical) [14]†	2.11	6.47	8.75	22.58	29.84	13.95	**10.74**	11.28	64.73	-	-	-
Ours	3.11	7.56	**6.44**	**15.55**	**23.84**	**11.30**	12.69	**9.06**	**51.86**	**6.62**	**25.03**	**8.69**

Table 3. Comparative quantitative evaluation on the ASAP test set. All prior methods produce quantized MIDI and require MuseScore 4 to perform typesetting and conversion to MusicXML. †: the reported metrics are slightly optimistic as some pieces of the test set appeared in the training data for subcomponents of this method only.

3.2 Metrics

To conduct fine-grained comparisons, we use both MUSTER [12, 18] and ScoreSimilarity [27, 42] as evaluation metrics for PM2S performance [2].

MUSTER especially focuses on high-level accuracy and rhythmic structure, with sub-metrics for note-level edit-distance ($\mathcal{E}_{\text{p}}$, $\mathcal{E}_{\text{miss}}$, $\mathcal{E}_{\text{extra}}$), rhythm correction ($\mathcal{E}_{\text{onset}}$), defined by the amount of scale and shift operations required to correctly align every note's onset with the ground truth sequence, and $\mathcal{E}_{\text{offset}}$, which measures the accuracy of the predicted note's musical durations. While edit-distance metrics primarily reflect the melodic correctness of a score, $\mathcal{E}_{\text{onset}}$ and $\mathcal{E}_{\text{offset}}$ serve as good indicators of rhythmic understanding and visual clarity of the resulting notation.

ScoreSimilarity also tracks edit-distances ($\mathcal{E}_{\text{miss}}$, $\mathcal{E}_{\text{extra}}$) but additionally allows the evaluation of notational details such as stem direction ($\mathcal{E}_{\text{stem}}$), pitch spelling ($\mathcal{E}_{\text{spell.}}$), or hand/staff assignment ($\mathcal{E}_{\text{staff}}$). We extend ScoreSimilarity to ornaments by adding F1-scores for grace, staccato, and trill. To harmonize the scores reported by both metrics, we opt to report normalized error scores and F1-scores instead of absolute error counts as originally proposed in [42].

	ScoreSimilarity [27, 42]					
	$\mathcal{E}_{\text{staff}}$	$\mathcal{E}_{\text{stem}}$	$\mathcal{E}_{\text{spell.}}$	$\mathbf{F1}_{\text{grace}}$	$\mathbf{F1}_{\text{staccato}}$	$\mathbf{F1}_{\text{trill}}$
SOTA [19, 43, 44]	6.86	26.74	9.69	-	-	-
Ours	**6.62**	**25.03**	**8.69**	**27.80**	**18.19**	**54.64**

Table 4. Predicting score ornaments and visual details.

Method	L	ScoreSimilarity [27, 42]				
		$\mathcal{E}_{\text{miss}}$	$\mathcal{E}_{\text{extra}}$	$\mathcal{E}_{\text{duration}}$	$\mathcal{E}_{\text{staff}}$	$\mathcal{E}_{\text{stem}}$
Suzuki [27]	12954	12.53	4.21	**0.53**	0.03	5.40
Octuple [28]	3697	3.56	4.74	17.51	16.34	30.94
Ours	**3697**	**2.64**	**0.40**	3.72	**0.01**	**1.54**
MIDI score [3]	-	3.04	4.64	13.63	3.41	-
MusicXML	-	0.00	0.00	0.00	0.00	0.00

Table 5. Comparison of score representation schemes by sequence lengths and representation error rates.

[2] MV2H [45] was also considered, but its alignment procedure was prohibitively slow on real-world scores with thousands of notes. Alignment is necessitated by the lack of one-to-one correspondence labels.

[3] The MIDI files were created from MusicXML with MuseScore 4.0.

3.3 Comparative experiments

PM2S. In Table 3, we compare our model to the best publicly available PM2S systems. Our baselines include the popular commercial programs MuseScore [43] and Finale [44], the strongest HMM-based approach [14], and the highest-performance deep learning model [19], which relies on neural beat tracking. Where necessary for evaluation, MuseScore 4 is employed to convert quantized score MIDI predictions to MusicXML. We also compare with an improved version of the reference implementation of [19], which removes the time-signature [4] and note-value prediction modules. However, as noted in Section 3.1.1, beat-tracking still struggles on real-world P-MIDI, lagging behind [14] and other options in rhythm quantization.

In contrast, our method predicts notation with significantly more accurate rhythm ($\mathcal{E}_{\text{onset/offset}}$), note values ($\mathcal{E}_{\text{offset/duration}}$), and fewer extraneous notes ($\mathcal{E}_{\text{extra}}$). In practice, this is reflected in better alignment of notes with barlines and more concise notation than alternative approaches. While all baselines pass the input pitch sequence directly to the output, our setup requires the model to rebuild the full sequence from scratch, leading to more missed notes ($\mathcal{E}_{\text{p/miss}}$). Decoupling the output pitch sequence from the input is key to our method, enabling training without one-to-one correspondences and predicting many-to-one relationships like trills. In fact, many 'misses' occur because our approach notated a trill where the ground truth score contains multiple alternating notes, with minimal impact on the resulting score's quality from a human perspective.

For sample scores and visual comparisons with baseline approaches, we refer to the supplementary material.

Notation details. To our knowledge, our method is the first PM2S system to predict note-level attributes beyond timing, pitch, and staff assignment. The model also estimates staccato, grace notes, and trill marks, which are crucial for human performers. Given the data imbalance – for instance, trills account for only 0.15% of notes – achieving high F1 scores is extremely challenging. Table 4 shows that our approach predicts more accurate stem directions, pitch-spelling, and staff assignments, while exhibiting relatively good performance on grace and trill notes.

Tokenization scheme. We evaluate our MusicXML to-

[4] We found that assuming a fixed $\frac{4}{4}$ time signature improves results.

	Modification	MUSTER [14]						ScoreSimilarity [27, 42]				
		$\mathcal{E}_{\text{p}}$	$\mathcal{E}_{\text{miss}}$	$\mathcal{E}_{\text{extra}}$	$\mathcal{E}_{\text{onset}}$	$\mathcal{E}_{\text{offset}}$	$\mathcal{E}_{\text{avg}}$	$\mathcal{E}_{\text{miss}}$	$\mathcal{E}_{\text{extra}}$	$\mathcal{E}_{\text{duration}}$	$\mathcal{E}_{\text{staff}}$	$\mathcal{E}_{\text{stem}}$
1	no data augmentation	10.96	38.05	38.23	37.89	51.34	35.30	49.99	47.98	32.43	7.30	13.10
2	BiLSTM backbone	4.01	22.28	16.02	38.30	60.30	28.12	26.75	16.29	55.46	7.55	21.08
3	BiGRU backbone	3.85	19.49	13.56	30.98	49.59	23.55	23.84	14.39	43.69	7.50	22.33
4	no beat-alignment	3.57	25.53	10.91	19.25	29.60	17.77	33.40	10.91	41.06	6.03	20.20
5	no transpose	4.86	12.04	9.89	19.36	29.66	15.16	17.99	12.76	50.54	7.92	**24.93**
6	no 24-div quantization	**3.02**	10.96	8.99	19.71	31.73	14.88	15.43	10.50	74.92	7.54	26.84
7	ALiBi pos. enc.	3.12	11.22	8.07	17.85	27.66	13.58	16.24	10.11	55.67	7.35	27.92
8	no surrogate pitch	4.11	9.83	8.20	16.89	26.66	13.14	15.21	10.79	**49.41**	6.81	25.84
9	no onset jitter	3.20	8.66	7.23	17.04	27.75	12.78	13.42	9.38	51.33	8.59	25.93
10	no tempo augmentation	3.18	8.74	7.21	17.25	27.45	12.76	13.60	9.50	54.20	7.78	26.95
11	no conditioning token	3.09	9.10	7.10	16.82	27.01	12.62	13.98	9.18	52.98	8.33	26.57
12	no duration jitter	3.39	8.53	7.32	16.54	26.92	12.54	13.56	9.59	50.68	8.19	27.69
13	sinusoidal pos. enc.	3.50	8.29	6.83	16.71	27.35	12.49	13.41	9.36	51.68	7.35	25.81
14	**Ours**	3.11	**7.56**	**6.44**	**15.55**	**23.84**	**11.30**	**12.69**	**9.06**	51.86	**6.62**	25.03

Table 6. Ablation study for key design decisions. Grayed out values do not reflect the true model performance as a large fraction of notes are misaligned during metric computations, leading to incorrect results. Rows are organized by $\mathcal{E}_{\text{avg}}$.

kenization against prior methods and score-derived MIDI files by converting ground-truth scores to a new format and then comparing the reconstructions to the originals.

Table 5 shows that our approach yields $3.5\times$ shorter sequence lengths than prior MusicXML tokenizations while maintaining more detail than alternatives. Furthermore, it highlights MIDI's shortcomings as a notation format; both ground-truth MIDI scores and MIDI-based tokenizations [28] exhibit lower fidelity than MusicXML-derived tokenizations and particularly high error rates for details like stem directions, which are not supported by MIDI.

3.4 Ablation study

Our ablation study in Table 6 shows the impact of key design choices.

Backbone architecture. The transformer architecture is much stronger than classic recurrent networks like bidirectional LSTMs [33] and GRUs [46] when trained on the same data (rows 3 & 2). We also evaluate the effectiveness of the conditioning token (row 11) and demonstrate the impact of feeding surrogate pitch information to the encoder for unpaired data (row 8).

Positional encoding. We compare rotary embeddings with standard sinusoidal embeddings [32] (row 13) and ALiBi [47] (row 7) and find that sinusoidal embeddings perform slightly weaker than rotary encoding while ALiBi yields significantly worse results.

Tokenization. We demonstrate the impact of our tokenization scheme's quantization by changing the encoding scheme to quantize durations by 32nd-divisions instead of 24th (row 6). This has a particularly strong impact on metrics for rhythm and note values ($\mathcal{E}_{\text{onset}}$, $\mathcal{E}_{\text{offset}}$, $\mathcal{E}_{\text{duration}}$).

Data augmentation & alignment. Data augmentation is crucial to the effectiveness of our approach (row 1). Rows 5, 9, 10, and 12 show that using all 4 augmentation types combined yields the best results. We also demonstrate that using our greedy beat-level note alignment algorithm significantly improves performance compared to unoptimized input and output sequence alignment (row 4).

3.5 Scaling

To assess model performance as datasets grow, we conduct experiments with varying amounts of paired and unpaired data (see Table 7). We observe a clear trend where increasing the amount of paired and unpaired data improves final performance across most metrics. The benefits of adding 10,000 unpaired scores are similar to expanding from 100 paired to 822 paired training pieces. While training on unpaired data is less sample-efficient, the labeling cost-savings may make it worthwhile nonetheless. The results suggest that our method is able to leverage additional data well and that training on larger (paired & unpaired) datasets could lead to significant further improvements.

Number of pieces		ScoreSimilarity [42]				
Paired	Unpaired	$\mathcal{E}_{\text{miss}}$	$\mathcal{E}_{\text{extra}}$	$\mathcal{E}_{\text{duration}}$	$\mathcal{E}_{\text{staff}}$	$\mathcal{E}_{\text{stem}}$
100	-	22.04	17.78	54.22	8.77	28.43
822	-	14.44	10.11	50.77	7.78	27.56
100	10,000	15.40	12.03	55.98	7.53	26.93
822	10,000	13.44	9.49	**49.18**	8.22	27.85
822	58,686	**12.69**	**9.06**	51.86	**6.62**	**25.03**

Table 7. The effects of training dataset size.

4. CONCLUSION

We presented a flexible, robust, and conceptually simple approach to convert P-MIDI into musical notation and showed that a standard sequence-to-sequence transformer model can outperform existing methods that benefit from extensive domain-specific optimizations. We also introduced a compact tokenization method for symbolic music data that is extensible to further notational elements in the future. Furthermore, we demonstrate that leveraging large unpaired training datasets can improve model performance and enhance the fidelity of predicted scores. Future efforts could add features like explicit key and time signature prediction, tempo marks, and expand to other musical genres, instruments, and notation systems.

5. ETHICS STATEMENT

The proposed system is primarily trained on classical piano music by European composers engraved in standard Western musical notation. While preliminary experiments show that the method generalizes out-of-genre to modern piano pop music, our approach nonetheless excludes a large body of musical work notated in different formats. Future work should aim to address this imbalance and create systems that can be useful to an even wider audience.

6. ACKNOWLEDGEMENTS

We would like to thank Andrew McLeod for his assistance and feedback on a draft of this work via the New-to-ISMIR paper mentoring program. This work was partially supported by the ERC Starting Grant SpatialSem (101076253).

7. REFERENCES

[1] N. Orio and D. Schwarz, "Alignment of monophonic and polyphonic music to a score," in *International Computer Music Conference (ICMC)*, 2001, pp. 1–1.

[2] C. Hawthorne, E. Elsen, J. Song, A. Roberts, I. Simon, C. Raffel, J. Engel, S. Oore, and D. Eck, "Onsets and frames: Dual-objective piano transcription," *arXiv preprint arXiv:1710.11153*, 2017.

[3] E. Benetos, S. Dixon, Z. Duan, and S. Ewert, "Automatic music transcription: An overview," *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 20–30, 2018.

[4] M. Good, "MusicXML for notation and analysis," *The virtual score: representation, retrieval, restoration*, vol. 12, no. 113–124, p. 160, 2001.

[5] P. Desain and H. Honing, "The quantization of musical time: A connectionist approach," *Computer Music Journal*, vol. 13, no. 3, pp. 56–66, 1989.

[6] C. Raphael, "Automated rhythm transcription," in *ISMIR*, vol. 2001, 2001, pp. 99–107.

[7] A. Cogliati, D. Temperley, and Z. Duan, "Transcribing human piano performances into music notation," in *ISMIR*, 2016, pp. 758–764.

[8] D. Temperley, "A unified probabilistic model for polyphonic music analysis," *Journal of New Music Research*, vol. 38, no. 1, pp. 3–18, 2009.

[9] H.-W. Nienhuys and J. Nieuwenhuizen, "LilyPond, a system for automated music engraving," in *Proceedings of the XIV Colloquium on Musical Informatics (XIV CIM 2003)*, vol. 1. Citeseer, 2003, pp. 167–171.

[10] E. Nakamura, N. Ono, and S. Sagayama, "Merged-output HMM for piano fingering of both hands," in *ISMIR*, 2014, pp. 531–536.

[11] E. Nakamura, K. Yoshii, and S. Sagayama, "Rhythm transcription of polyphonic piano music based on merged-output hmm for multiple voices," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, no. 4, pp. 794–806, 2017.

[12] E. Nakamura, E. Benetos, K. Yoshii, and S. Dixon, "Towards complete polyphonic music transcription: Integrating multi-pitch detection and rhythm quantization," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2018, pp. 101–105.

[13] E. Nakamura, K. Yoshii, and S. Dixon, "Note value recognition for piano transcription using markov random fields," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, no. 9, pp. 1846–1858, 2017.

[14] K. Shibata, E. Nakamura, and K. Yoshii, "Non-local musical statistics as guides for audio-to-score piano transcription," *Information Sciences*, vol. 566, pp. 262–280, 2021.

[15] R. G. C. Carvalho and P. Smaragdis, "Towards end-to-end polyphonic music transcription: Transforming music audio directly to a score," in *2017 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*. IEEE, 2017, pp. 151–155.

[16] M. A. Román, A. Pertusa, and J. Calvo-Zaragoza, "Data representations for audio-to-score monophonic music transcription," *Expert Systems with Applications*, vol. 162, p. 113769, 2020.

[17] F. Foscarin, N. Audebert, and R. Fournier-S'Niehotta, "PKSpell: Data-driven pitch spelling and key signature estimation," *arXiv preprint arXiv:2107.14009*, 2021.

[18] Y. Hiramatsu, E. Nakamura, and K. Yoshii, "Joint estimation of note values and voices for audio-to-score piano transcription," in *ISMIR*, 2021, pp. 278–284.

[19] L. Liu, Q. Kong, G. Morfi, E. Benetos *et al.*, "Performance MIDI-to-score conversion by neural beat tracking," 2022.

[20] T. Cheng and M. Goto, "Transformer-based beat tracking with low-resolution encoder and high-resolution decoder," in *ISMIR 2023 Hybrid Conference*, 2023.

[21] C. Walshaw, "The abc music standard 2.1," *URL: http://abcnotation. com/wiki/abc: standard: v2*, vol. 1, 2011.

[22] D. Huron, "Music information processing using the Humdrum toolkit: Concepts, examples, and lessons," *Computer Music Journal*, vol. 26, no. 2, pp. 11–26, 2002.

[23] R. Kelz, M. Dorfer, F. Korzeniowski, S. Böck, A. Arzt, and G. Widmer, "On the potential of simple framewise approaches to piano transcription," *arXiv preprint arXiv:1612.05153*, 2016.

[24] Y.-S. Huang and Y.-H. Yang, "Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions," in *Proceedings of the 28th ACM international conference on multimedia*, 2020, pp. 1180–1188.

[25] C. Hawthorne, I. Simon, R. Swavely, E. Manilow, and J. Engel, "Sequence-to-sequence piano transcription with transformers," in *Proc. of the 22nd Int. Society for Music Information Retrieval Conf.*, Online, 2021.

[26] J. Gardner, I. Simon, E. Manilow, C. Hawthorne, and J. Engel, "MT3: Multi-task multitrack music transcription," *arXiv preprint arXiv:2111.03017*, 2021.

[27] M. Suzuki, "Score transformer: Generating musical score from note-level representation," in *Proceedings of the 3rd ACM International Conference on Multimedia in Asia*, 2021, pp. 31:1–31:7.

[28] M. Zeng, X. Tan, R. Wang, Z. Ju, T. Qin, and T.-Y. Liu, "MusicBERT: Symbolic music understanding with large-scale pre-training," *arXiv preprint arXiv:2106.05630*, 2021.

[29] H.-W. Dong, K. Chen, S. Dubnov, J. McAuley, and T. Berg-Kirkpatrick, "Multitrack music transformer," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[30] A. Lv, X. Tan, P. Lu, W. Ye, S. Zhang, J. Bian, and R. Yan, "GETMusic: Generating any music tracks with a unified representation and diffusion framework," *arXiv preprint arXiv:2305.10841*, 2023.

[31] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[33] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[34] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[35] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[36] N. Shazeer, "Glu variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[37] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.

[38] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," *Advances in neural information processing systems*, vol. 28, 2015.

[39] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[40] V. Emiya, N. Bertin, B. David, and R. Badeau, "MAPS - A piano database for multipitch estimation and automatic transcription of music," 2010.

[41] F. Foscarin, A. Mcleod, P. Rigaux, F. Jacquemard, and M. Sakai, "ASAP: A dataset of aligned scores and performances for piano transcription," in *International Society for Music Information Retrieval Conference*, 2020, pp. 534–541.

[42] A. Cogliati and Z. Duan, "A metric for music notation transcription accuracy," in *ISMIR*, 2017, pp. 407–413.

[43] MuseScore B.V., "MuseScore: A free and open-source music notation software," https://musescore.org/, 2002, accessed: 2024-02-28.

[44] MakeMusic, Inc., "Finale version 27," https://www.finalemusic.com/, 1988, accessed: 2024-02-28.

[45] A. McLeod, "Evaluating non-aligned musical score transcriptions with MV2H," *arXiv preprint arXiv:1906.00566*, 2019.

[46] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.

[47] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," *arXiv preprint arXiv:2108.12409*, 2021.